Instruction Format Specification: Two-Pass IA-32 (x86 32-Bit) Assembler

1. Instruction Byte Layout Diagram

An IA-32 instruction (excluding prefixes) consists of up to 5 functional fields ordered sequentially in memory. All fields except the Opcode are optional depending on the instruction type and addressing mode.

Opcode	**ModR/M**	**SIB**	**Displacement**	**Immediate**
1–2 Bytes	**0–1 Byte**	**0–1 Byte**	**0, 1, or 4 Bytes**	**0, 1, 2, or 4 Bytes**
Mandatory instruction identifier	Specifies addressing mode and operands	Scale-Index-Base byte for complex memory	8-bit or 32-bit address offset (Little-Endian)	Constant data operand (Little-Endian)

2. Register Encoding Lookup Table

The 3-bit binary identifier assigned to each 32-bit general-purpose register:

Register	**Binary Code**	**Hex Code**
EAX	000	0
ECX	001	1
EDX	010	2
EBX	011	3
ESP	100	4
EBP	101	5
ESI	110	6
EDI	111	7

3. Addressing Bytes Specification (ModR/M & SIB)

ModR/M Byte Breakdown

The ModR/M byte encodes operand types and addressing modes.

Bit: [7 6] [5 4 3] [2 1 0]
Field: Mod Reg R/M

- **Mod (Bits 7–6):**
  - 00: Memory mode without displacement (unless R/M = 101, which specifies 32-bit displacement).
  - 01: Memory mode with 8-bit signed displacement (disp8).
  - 10: Memory mode with 32-bit signed displacement (disp32).
  - 11: Register-to-Register mode (both operands are registers).
- **Reg (Bits 5–3):** Specifies the source/destination register or an opcode extension.
- **R/M (Bits 2–0):** Specifies the register operand or base memory register. If R/M = 100 in memory mode (Mod != 11), an **SIB byte** must follow.

SIB (Scale-Index-Base) Byte Breakdown

Used for scaled-indexed memory addressing (e.g., [Base + Index * Scale + Displacement]).

Bit: [7 6] [5 4 3] [2 1 0]
Field: Scale Index Base

- **Scale (Bits 7–6):** Multiplier for the Index register (00 = ×1, 01 = ×2, 10 = ×4, 11 = ×8).
- **Index (Bits 5–3):** 3-bit code of the index register (ESP / 100 cannot be used as an index).
- **Base (Bits 2–0):** 3-bit code of the base register.

4. Core Opcode Map

Mnemonic	Operands	Opcode (Hex)	Encoding Format
NOP	None	90	Single byte
RET	None	C3	Single byte
MOV	reg32, reg32	89 /r	ModR/M (Mod=11)
MOV	reg32, [mem]	8B /r	ModR/M + optional SIB/Disp
MOV	reg32, imm32	B8 + rd	Opcode + 32-bit Immediate
ADD	reg32, reg32	01 /r	ModR/M (Mod=11)
ADD	reg32, imm32	81 /0	ModR/M (Reg=000) + Imm32
SUB	reg32, reg32	29 /r	ModR/M (Mod=11)
JMP	rel8	EB cb	Opcode + 8-bit relative offset
JNE	rel8	75 cb	Opcode + 8-bit relative offset

5. Directives & Memory Allocation Rules

Directive	Purpose	Size	Little-Endian Alignment
DB	Define Byte	1 Byte	Stores 8-bit literal byte directly.
DW	Define Word	2 Bytes	Reverses byte order (LSB first, MSB last).
DD	Define Double Word	4 Bytes	Reverses 4-byte integer/address in memory.
SECTION .text	Code Segment	N/A	Sets symbol table location counter to .text origin.

SECTION .data	Data Segment	N/A	Allocates memory address space for variables.

6. Immediates, Displacements & Jump Offsets

1. **Little-Endian Rule:** Values larger than 1 byte are stored with the least significant byte (LSB) at the lowest memory address.

- *Example:* 0x12345678 → Byte sequence: 78 56 34 12.

2. **EIP-Relative Jump Calculation:**

text{Relative Offset} = \text{Target Address} - (\text{Instruction Address} + \text{Instruction Size})

Negative relative offsets are formatted using 8-bit or 32-bit Two's Complement notation.

7. Sample Code Translation Walkthrough

Assembly Input Snippet

code snippet

```
SECTION .text
MOV EAX, 0x00000005
ADD EAX, EBX
MOV ECX, [EBX + ESI*4 + 0x10]
JNE -5
```

Line-by-Line Encoding Breakdown

Line 1: MOV EAX, 0x00000005

- **Opcode:** B8 + rd where rd = EAX (000) → **B8**
- **Immediate (32-bit):** 0x00000005 in Little-Endian → **05 00 00 00**
- **Machine Code:** B8 05 00 00 00 (5 bytes)

Line 2: ADD EAX, EBX

- **Opcode:** 01 /r → **01**
- **ModR/M Byte:**
  - Mod = 11 (Register to Register)
  - Reg = 011 (EBX - Source)

- R/M = 000 (EAX - Destination)
- Binary: 11 011 000 = 0xD8

- **Machine Code:** 01 D8 (2 bytes)

Line 3: MOV ECX, [EBX + ESI*4 + 0x10]

- **Opcode:** 8B /r → **8B**
- **ModR/M Byte:**
- Mod = 01 (Memory with 8-bit displacement 0x10)
- Reg = 001 (ECX - Destination)
- R/M = 100 (SIB byte follows)
- Binary: 01 001 100 = 0x4C
- **SIB Byte:**
- Scale = 10 (×4)
- Index = 110 (ESI)
- Base = 011 (EBX)
- Binary: 10 110 011 = 0xB3
- **Displacement:** 0x10 → **10**
- **Machine Code:** 8B 4C B3 10 (4 bytes)

Line 4: JNE -5

- **Opcode:** 75
- **Relative Offset:** -5 in 8-bit Two's Complement → **FB**
- **Machine Code:** 75 FB (2 bytes)

Final Encoded Hexadecimal Stream

Assembly Line	**Address Offset**	**Machine Code (Hex)**	**Byte Length**
MOV EAX, 0x00000005	0x00	B8 05 00 00 00	5 Bytes
ADD EAX, EBX	0x05	01 D8	2 Bytes

MOV ECX, [EBX + ESI*4 + 0x10]	0x07	8B 4C B3 10	4 Bytes
JNE -5	0x0B	75 FB	2 Bytes

Complete Binary Output Sequence:

B8 05 00 00 00 01 D8 8B 4C B3 10 75 FB

Repo link :

https://github.com/Rmkhose022/Cs602